

数据结构与算法 作业9: Mock Exam立冬 (🌲)

2025 fall

Nov月考: AC2

说明:

1. 考试题目都在“题库（包括计概、数算题目）”里面，按照数字题号能找到，可以重新提交。作业中提交自己最满意版本的代码和截图。
2. 解题与记录：对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。
3. 提交安排：提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。
4. 延迟提交：如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

M02255: 重建二叉树

<http://cs101.openjudge.cn/practice/02255/>

思路：耗时30min。月考的时候没注意看，直接跳过了。后面回来很快就AC了，很后悔。这题其实比较简单，只要利用前序遍历中，第一个节点必定是root，再利用中序遍历中，root节点左侧是左子树，右侧是右子树来确定左右子树的元素，最后递归左、右子树的前序遍历和中序遍历，返回root作为子树的节点即可。

代码:

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

通过前序遍历的list，确定子树的中序遍历对应的前序遍历

```

def node_in_pre_and_mid(mid, parent_pre):
    mid_set = set(mid)
    pre = []
    for i in parent_pre:
        if i in mid_set:
            pre.append(i)
    return pre

# 用于实现建树思路的函数
def build_tree(pre: list, mid: list):
    root = TreeNode(pre[0])
    index = 0
    for i in range(len(mid)):
        if mid[i] == pre[0]:
            index = i
            break
    left_mid = mid[:index]
    right_mid = mid[index + 1:]
    if len(left_mid) > 0:
        left_pre = node_in_pre_and_mid(left_mid, pre)
        root.left = build_tree(left_pre, left_mid)
    if len(right_mid) > 0:
        right_pre = node_in_pre_and_mid(right_mid, pre)
        root.right = build_tree(right_pre, right_mid)

    return root

```

```

# 后序遍历
def postorder(node: TreeNode, trace: list):
    if node is not None:
        trace = postorder(node.left, trace)
        trace = postorder(node.right, trace)
        trace.append(node.val)
    return trace

```

```

def main():
    while 1:
        try:
            pre, mid = [list(x) for x in input().split()]
            root = build_tree(pre, mid)
            print(''.join(postorder(root, trace=[])))
        except EOFError:
            break

```

```
if __name__ == '__main__':
    main()
```

代码运行截图 (至少包含有"Accepted")

#50756244提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def node_in_pre_and_mid(mid, parent_pre):
    mid_set = set(mid)
    pre = []
    for i in parent_pre:
        if i in mid_set:
            pre.append(i)
```

基本信息

#: 50756244
题目: 02255
提交人: 22n2200011816(略约横溪)
内存: 3684kB
时间: 23ms
语言: Python3
提交时间: 2025-11-08 18:44:06

M02774: 木材加工

<http://cs101.openjudge.cn/practice/02774/>

思路: 耗时35 min。月考AC。看到代码的时候立刻意识到这是很经典的二分查找的题目。将左边界设为0, 右边界为最长的木板长度, 随后每块木板除以二分查找的mid获得每块可以切出的木板块数并相加, 如果小于所需的木板数量, 说明现在mid过长, 右边界定为mid的值; 如果大于等于所需木板数量, 说明木板过短/刚好, 左边界定为mid的值, 直到left > right。

代码:

```
def minicost(num, wood_need, cost_list):
    left = 0 # 使用二分查找获得合适的length
    right = max(cost_list)
    while left <= right:
        mid = (right + left) // 2
        wood_needed_in_mid = 0
        if mid == 0:
            return 0
        for j in range(num):
            wood_needed_in_mid += (cost_list[j]//mid)
        #print(mid, wood_needed_in_mid)
        if wood_needed_in_mid < wood_need:
            right = mid - 1
        elif wood_needed_in_mid >= wood_need:
            left = mid + 1
    return left - 1

def main():
    n, k = (int(x) for x in input().split())
```

```

cost_list = []
for i in range(n):
    cost_list.append(int(input()))
cost = minicost(n, k, cost_list)
print(cost)

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

M02788: 二叉树 (2)

<http://cs101.openjudge.cn/practice/02788/>

思路：耗时1 h。月考AC。主要利用了数学法解出了题目。

由于树的构建规则，如果求出子节点m和总节点数n的 $2^x + y$ 的形式，则节点m为深度 x_m 上的第 y_m 个节点。

1. 因此，由于所处深度 x_m 和 x_n 的差异，m就有 $2^{x_n-x_m} - 1$ 个子节点；
2. 接着再考察y的影响：深度 x_m 上的第 y_m 个节点在深度 x_n 上最左的子节点的 $y'_n = y_m * 2^{x_n-x_m}$ ，在深度 x_n 上，m又有 $y_n - y_m * 2^{x_n-x_m} + 1$ 个子节点，注意子节点最小不小于0，最大不超过 $2^{x_n-x_m}$ ，最后将两部分的值相加即可。

代码：

```

def two_power(n: int):
    power = 0
    while 1:
        if n // 2**power > 1:
            power += 1
        else:
            return power

def main():
    while 1:
        m, n = [int(x) for x in input().split()]
        if m == 0 and n == 0:
            break
        power_m, power_n = two_power(m), two_power(n)
        remain_m, remain_n = m % (2**power_m), n % (2**power_n)
        part1 = 2**(power_n - power_m) - 1
        part2 = max(min(2**(power_n - power_m), remain_n - remain_m*2**
(power_n - power_m) + 1), 0)
        ans = part1 + part2
        print(ans)

```

```
if __name__ == '__main__':
    main()
```

代码运行截图 (至少包含有"Accepted")

#50744756提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
def two_power(n: int):
    power = 0
    while 1:
        if n // 2**power > 1:
            power += 1
        else:
            return power

def main():
    while 1:
        m, n = [int(x) for x in input().split()]
        if m == 0 and n == 0:
            break
        power_m, power_n = two_power(m), two_power(n)
        remain_m, remain_n = m % (2**power_m), n % (2**power_n)
```

基本信息

#: 50744756
题目: 02788
提交人: 22n2200011816(略约横溪)
内存: 3600kB
时间: 65ms
语言: Python3
提交时间: 2025-11-07 20:52:43

M04081: 树的转换

<http://cs101.openjudge.cn/practice/04081/>

思路: 耗时1 h

1. 首先按照题目的要求建树: 题目为前序遍历, 在输入的字符串中, 如果当前的index为d, 代表深度+1, 创建一个新的node并向下递归; 否则, 就退出当前深度, 返回node, 并将返回的node添加到children里面。
 2. 其次转化为二叉树: 使用队列的方式, 将所有children加入queue中, 第一个弹出的节点进入firstchild, 其他节点在弹出后变为bro; 弹出的同时递归弹出节点的子节点即可。
 3. 计算深度: 分而治之, 比较左右两侧的深度返回最大值
- 这题思路其实不难想, 但是由于我不会数学上更直观的解法, 只能手搓建树代码和树转化为大儿子-兄弟树, 还要计算深度, 整得代码显得又臭又长。

代码

```
from collections import deque

class TreeNode:
    def __init__(self, val=0):
        self.val = val
        self.children = []

class binaryTreeNode:
    def __init__(self, val=None, firstChild=None, bro=None):
        self.val = val
        self.firstChild = firstChild
```

```
self.bro = bro
```

```
# 按照输入的规则建树
```

```
def dfs(s: str, node: TreeNode, index: int):  
    while index < len(s):  
        if s[index] == 'd':  
            temp_node, index = dfs(s, TreeNode(index+1), index + 1)  
            node.children.append(temp_node)  
        else:  
            index += 1  
        return node, index  
    return node, index
```

```
# 树的转换：应用队列，对于子树，第一个节点进入firstchild，其他节点在弹出后变为bro
```

```
def tree_to_binarytree(node: TreeNode):  
    binarynode = binaryTreeNode(node.val)  
    if len(node.children) > 0:  
        q = deque(node.children)  
        temp_node_0 = tree_to_binarytree(q.popleft())  
        binarynode.firstChild = temp_node_0  
        for _ in range(len(node.children) - 1):  
            temp_node_1 = tree_to_binarytree(q.popleft())  
            temp_node_0.bro = temp_node_1  
            temp_node_0 = temp_node_1  
  
    return binarynode
```

```
# 计算深度，比较左右两侧的深度返回最大值
```

```
def deep(node):  
    if node is None:  
        return 0  
    if isinstance(node, TreeNode):  
        depth_list = []  
        for i in node.children:  
            depth_list.append(deep(i))  
        if len(depth_list) > 0:  
            return max(depth_list) + 1  
        else:  
            return 0  
    else:  
        left_deep = deep(node.firstChild)  
        right_deep = deep(node.bro)  
        return max(left_deep, right_deep) + 1
```

```
def main():  
    s = input()
```

```

index = 0
root, _ = dfs(s, TreeNode(index), index)
depth1 = deep(root)

binaryroot = tree_to_binarytree(root)
depth2 = deep(binaryroot) - 1

print(f'{depth1} => {depth2}')

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

#50745973提交状态

查看 提交 统计 提问

状态: Accepted

源代码

```

from collections import deque

class TreeNode:
    def __init__(self, val=0):
        self.val = val
        self.children = []

class binaryTreeNode:
    def __init__(self, val=None, firstChild=None, bro=None):
        self.val = val
        self.firstChild = firstChild
        self.bro = bro

def dfs(s: str, node: TreeNode, index: int):
    while index < len(s):
        if s[index] == 'd':
            temp_node, index = dfs(s, TreeNode(index+1), index + 1)

```

基本信息

#: 50745973
 题目: 04081
 提交人: 22n2200011816(略约横溪)
 内存: 3748kB
 时间: 22ms
 语言: Python3
 提交时间: 2025-11-07 22:17:30

M04117: 简单的整数划分问题

dfs, dp, <http://cs101.openjudge.cn/practice/04117/>

思路: 耗时3 h。这道题想了很久, 无奈对这种数学类的题目真的没想出来递推公式, 无奈看了题解。

对n整数的n划分, 首先等价于n整数的n-1划分再加一; 其次, 对n整数的m划分, 如果m>n, 由于不可能划分出负数, 其实就是n整数的n划分; 如果m<n, 那么分成两种情况: 1、如果划分中含有m, 则在含有一个m的情况下, 剩下部分为n-m整数的m划分, 2、如果划分中不含m, 则为n函数的m-1划分。在这个递推公式下, 如此就可以用dfs方法或dp方法求解。

最后, 这个题目的输入要用try-except的方式读取, 我一开始没有注意, 在测试数据的迷惑下以为只输入一组数据, 浪费了非常久的时间

代码

DFS:

```

def dfs(m, n):
    if n == 1 or m == 1:

```

```

        return 1
    elif n == m:
        return dfs(n-1, n) + 1
    elif n > m:
        return dfs(m, n-m) + dfs(m-1, n)
    elif n < m:
        return dfs(n, n)

def main():
    while 1:
        try:
            n = int(input())
            print(dfs(n, n))
        except EOFError:
            break

if __name__ == '__main__':
    main()

```

DP:

```

def main():
    while 1:
        try:
            n = int(input())
            dp = [[1 for _ in range(n + 1)] for _ in range(n + 1)]
            for i in range(1, n + 1):
                for j in range(1, n + 1):
                    if i == 1 or j == 1:
                        dp[i][j] = 1
                    elif i == j:
                        dp[i][j] = dp[i][j - 1] + 1
                    elif i > j:
                        dp[i][j] = dp[i - j][j] + dp[i][j - 1]
                    elif i < j:
                        dp[i][j] = dp[i][i]
            print(dp[n][n])
        except EOFError:
            break

if __name__ == '__main__':
    main()

```


#50754538提交状态

查看 提交 统计 提问

状态: Accepted

源代码

```
def dfs(m, n):
    if n == 1 or m == 1:
        return 1
    elif n == m:
        return dfs(n-1, n) + 1
    elif n > m:
        return dfs(m, n-m) + dfs(m-1, n)
    elif n < m:
        return dfs(n, n)

def main():
    while 1:
```

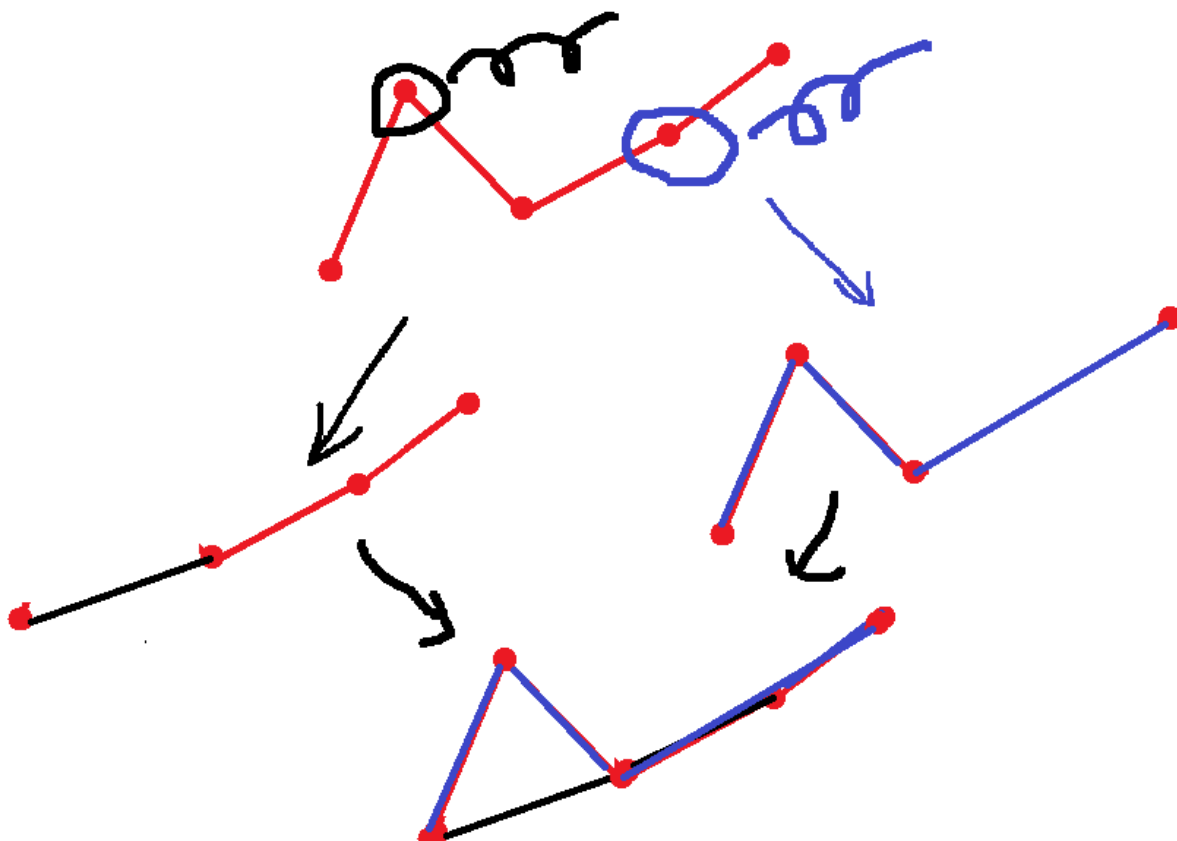
基本信息

#: 50754538
 题目: 04117
 提交人: 22n2200011816(略约横溪)
 内存: 5572kB
 时间: 32ms
 语言: Python3
 提交时间: 2025-11-08 16:59:17

M04137:最小新整数

monotonous-stack, <http://cs101.openjudge.cn/practice/04137/>

思路：耗时30 min。对一组数，如果要去掉某几个数后，组成的新数最小，就要保证新的数从左到右各位次尽量保证单调递增的形式，否则就会导致中间的“凸起”使结果并非最优解（下图）。为了保证单调递增的形式，就需要构建monotonic_stack。具体而言，将数依次压入栈中，如果某位数小于栈尾，则对栈进行pop消除“凸起”，并消耗一次去除次数。如果最后消除次数没有用完，则继续pop栈尾即可。



代码

```
def increasing_monotonic_stack(nums, k):
    stack = []
    count = 0
    for i in range(len(nums)):
```

```

        while stack and int(stack[-1]) > int(nums[i]):
            stack.pop()
            count += 1
        if count == k:
            stack.append(nums[i:])
            break
        stack.append(nums[i])

    for _ in range(k - count):
        stack.pop()
    return stack

def main():
    t = int(input())
    for _ in range(t):
        n, k = input().split()
        result = increasing_monotonic_stack(n, int(k))
        print(' '.join(result))

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#50755057提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

def increasing_monotonic_stack(nums, k):
    stack = []
    count = 0
    for i in range(len(nums)):
        while stack and int(stack[-1]) > int(nums[i]):
            stack.pop()
            count += 1
        if count == k:
            stack.append(nums[i:])
            break
        stack.append(nums[i])

    for _ in range(k - count):
        stack.pop()
    return stack

```

基本信息

#: 50755057
 题目: 04137
 提交人: 22n2200011816(略约横溪)
 内存: 3628kB
 时间: 20ms
 语言: Python3
 提交时间: 2025-11-08 17:22:19

2. 学习总结和收获

这周月考由于在外地无法现场参与，只能用自己的设备进行考试，最后AC2了，比上次月考的AC1有了成倍的进步。由于期中考试，对树的额外练习其实没有很多，但是也能感受到经过多周的训练，对“树”这种结构的有了比较深刻的认识，对遍历的理解更深刻、对树相关的题目也比较有思路了。

最后想询问一下老师：由于第一周尚未选课，没有参与课堂，对期末考试的要求不清晰。在期

末机考中，除了cheating paper和草稿纸，允许使用百度或在openjudge/leetcode中copy自己写过的代码吗？